

Prompting Fundamentals on a Real Task

TensorFlow / Python Runtime Debugging

Source task	FL-01 target: Debugging a complex TensorFlow/software issue
Goal	Improve a real debugging prompt one technique at a time
Runs	6 total: naive baseline + 5 iterations
Final check	Same final prompt run on Claude and ChatGPT

I chose this task because it is part of my actual TensorFlow open-source workflow. “Done well” means identifying the root cause, implementing the right fix, passing the relevant build/tests, and confirming the original failure no longer reproduces. The point of the exercise was to improve the quality of the debugging conversation without changing several prompt ingredients at once.

Iteration Overview

Run	Named technique	Main observed effect
Baseline	None	Too vague to diagnose anything; the model asked for basic missing details.
V1	Role assignment	The intake questions became more structured, but there was still no diagnosis.
V2	Context and motivation	The questions became specific to Python 3.14 free-threading, GIL behavior, crashes, races, and deadlocks.
V3	Few-shot examples	The model adopted a symptom -> mechanism -> verification-step analysis style.
V4	Output structure	The response separated likely cause, evidence, verification, “do not change yet,” and next action.
V5	Step decomposition	The model enforced evidence-first ordering and refused to skip from missing reproduction data to a patch.

Important observation: more prompt structure did not create missing evidence. The strongest later prompts improved discipline and reviewability, but neither model could identify a real root cause without an actual failing command, traceback, hang, or reproducer.

Baseline - Naive Prompt

Help me debug this TensorFlow issue.

Representative output

“Could you share a bit more detail - the error message or traceback, the relevant code snippet, and what you are trying to do?”

Observed output difference

The model could not begin debugging and asked for the traceback, code, TensorFlow version, and basic task context.

What still failed

The prompt was too vague to produce any technical diagnosis.

Why the next change

Add a role so the model approaches the problem as a TensorFlow/Python runtime debugging task rather than a generic support question.

Version 1 - Role Assignment

Act as a senior TensorFlow and Python runtime engineer.

Help me debug this TensorFlow issue.

Representative output

The response asked for a concrete debugging bundle: full traceback, reproducing code, environment details, installation method, and the operation that failed.

Observed output difference

The intake became more structured and professional. The model asked for the exact artifacts a runtime engineer would normally need.

What still failed

It still could not diagnose anything because the prompt contained no actual technical context.

Why the next change

Add context and motivation so the model knows this is specifically about TensorFlow on Python 3.14 free-threaded mode and why root-cause accuracy matters.

Version 2 - Context and Motivation

Act as a senior TensorFlow and Python runtime engineer.

Help me debug this TensorFlow issue.

Context:

I am working on TensorFlow compatibility with Python 3.14 free-threaded mode. The issue may involve native Python/C++ extension code, GIL assumptions, critical sections, deadlocks, race conditions, or build/runtime compatibility. I need to identify the real root cause before changing code because the fix may affect TensorFlow's runtime behavior and upstream contribution quality.

Representative output

Claude shifted from generic TensorFlow intake questions to free-threaded runtime details: Python 3.14t status, GIL state, crash vs. deadlock vs. race behavior, native-extension compatibility, thread dumps, and minimal reproduction.

Observed output difference

The response became technically targeted to Python 3.14t and the TensorFlow Python/C++ boundary instead of treating the problem as ordinary model training or installation support.

What still failed

It still had no concrete failure artifact and produced a fairly long checklist rather than the exact diagnostic style I wanted.

Why the next change

Add few-shot examples that demonstrate the analysis pattern I want: symptom, likely mechanism, then the smallest verification step before a patch.

Version 3 - Few-Shot Examples

Act as a senior TensorFlow and Python runtime engineer.

Help me debug this TensorFlow issue.

Context:

I am working on TensorFlow compatibility with Python 3.14 free-threaded mode. The issue may involve native Python/C++ extension code, GIL assumptions, critical sections, deadlocks, race conditions, or build/runtime compatibility. I need to identify the real root cause before changing code because the fix may affect TensorFlow's runtime behavior and upstream contribution quality.

Use these examples as the style of analysis I want:

Example 1:

Symptom: importing a native extension causes the GIL to become enabled in a free-threaded Python process.

Useful analysis: identify which extension initialization path can trigger GIL fallback, point to the relevant native boundary or module initialization behavior, explain why it happens, then suggest the smallest verification step before proposing a patch.

Example 2:

Symptom: a multithreaded test hangs only with Python 3.14 free-threaded mode.

Useful analysis: treat it first as a possible synchronization or critical-section problem, identify the threads or locks that need inspection, request a thread dump or minimal reproducer, and distinguish a deadlock from a race condition before recommending code changes.

Help me debug the issue using that same level of specificity.

Representative output

Claude used the examples to propose two plausible branches - silent GIL re-enable on import and a multithreaded deadlock/race - and attached a concrete verification step to each instead of only asking for more information.

Observed output difference

The analysis became more concrete. The model copied the useful reasoning pattern from the examples: symptom -> mechanism -> verification.

What still failed

It still had to guess between candidate failure modes because no real symptom had been supplied, and the response mixed diagnosis, explanation, and questions together.

Why the next change

Add an output structure so evidence, hypotheses, verification, and next actions are easier to inspect.

Version 4 - Output Structure

Act as a senior TensorFlow and Python runtime engineer.

Help me debug this TensorFlow issue.

Context:

I am working on TensorFlow compatibility with Python 3.14 free-threaded mode. The issue may involve native Python/C++ extension code, GIL assumptions, critical sections, deadlocks, race conditions, or build/runtime compatibility. I need to identify the real root cause before changing code because the fix may affect TensorFlow's runtime behavior and upstream contribution quality.

Use these examples as the style of analysis I want:

Example 1:

Symptom: importing a native extension causes the GIL to become enabled in a free-threaded Python process.

Useful analysis: identify which extension initialization path can trigger GIL fallback, point to the relevant native boundary or module initialization behavior, explain why it happens, then suggest the smallest verification step before proposing a patch.

Example 2:

Symptom: a multithreaded test hangs only with Python 3.14 free-threaded mode.

Useful analysis: treat it first as a possible synchronization or critical-section problem, identify the threads or locks that need inspection, request a thread dump or minimal reproducer, and distinguish a deadlock from a race condition before recommending code changes.

Format the response as:

1. Likely failure mode
2. Why it is plausible
3. Evidence I should collect
4. Smallest verification step
5. What not to change yet
6. Next action

Representative output

The response separated the investigation into two candidate failure modes, the evidence needed for each, one minimal command to distinguish them, changes to avoid before verification, and the next action.

Observed output difference

The result became much easier to review and act on. Hypotheses, evidence, and next steps were separated instead of being buried in a narrative.

What still failed

The model was still reasoning from hypothetical branches because no concrete reproducer was present. Better formatting did not solve the missing-evidence problem.

Why the next change

Add step decomposition so the model is required to reproduce, classify, isolate, falsify, and only then consider a patch.

Version 5 - Step Decomposition

Act as a senior TensorFlow and Python runtime engineer.

Help me debug this TensorFlow issue.

Context:

I am working on TensorFlow compatibility with Python 3.14 free-threaded mode. The issue may involve native Python/C++ extension code, GIL assumptions, critical sections, deadlocks, race conditions, or build/runtime compatibility. I need to identify the real root cause before changing code because the fix may affect TensorFlow's runtime behavior and upstream contribution quality.

Use these examples as the style of analysis I want:

Example 1:

Symptom: importing a native extension causes the GIL to become enabled in a free-threaded Python process.

Useful analysis: identify which extension initialization path can trigger GIL fallback, point to the relevant native boundary or module initialization behavior, explain why it happens, then suggest the smallest verification step before proposing a patch.

Example 2:

Symptom: a multithreaded test hangs only with Python 3.14 free-threaded mode.

Useful analysis: treat it first as a possible synchronization or critical-section problem, identify the threads or locks that need inspection, request a thread dump or minimal reproducer, and distinguish a deadlock from a race condition before recommending code changes.

Format the response as:

1. Likely failure mode
2. Why it is plausible
3. Evidence I should collect
4. Smallest verification step
5. What not to change yet
6. Next action

Work through the debugging process in this order:

1. Reproduce the failure with the smallest possible test.
2. Classify it as import/GIL behavior, build failure, crash, deadlock, race condition, or incorrect result.
3. Isolate the Python/C++ or runtime boundary involved.
4. Form one primary hypothesis supported by the available evidence.
5. Give the smallest test that can falsify that hypothesis.
6. Only if the hypothesis survives verification, propose the smallest code change.
7. State which build or tests should be run afterward to prove the original problem is fixed and no obvious regression was introduced.

Do not skip ahead to a code fix if an earlier step is still unverified.

Representative output

Claude explicitly stopped at step 1: without an actual symptom or reproducer, it would not proceed to classification, hypothesis, or code changes. It asked for the smallest GIL/import verification command or a real crash/hang artifact.

Observed output difference

The response became more disciplined about process order. The model enforced the evidence-first sequence instead of producing more speculative technical detail.

What still failed

This did not improve the technical diagnosis itself because the prompt still contained no real failing test, traceback, hang, or build error.

Why the next change

This was the final iteration. The useful lesson was that decomposition can prevent premature fixes, but it cannot replace missing evidence.

Cross-Model Comparison - Final Prompt

I ran the same Version 5 prompt on Claude and ChatGPT. Claude was used on the free plan and ChatGPT on the Plus plan, so this is a practical comparison of the outputs I actually received, not a controlled model-tier benchmark.

Aspect	Claude	ChatGPT
Tone	More conservative and concise. It stopped quickly when the missing reproducer blocked step 1.	More explanatory and playbook-like. It stayed evidence-first but expanded the investigation framework.
Accuracy / focus	Correctly treated the lack of a concrete symptom as the main blocker and avoided pretending to know the root cause.	Also avoided claiming a root cause, while mapping likely failure classes and the Python -> native runtime boundary in more detail.
Structure	Followed the requested order but mostly used it to explain why later steps could not be executed yet.	Turned the order into a fuller debugging workflow: classify, isolate boundary, form one hypothesis, falsify it, then patch.
Useful detail	Gave one immediate verification command and asked for a real artifact.	Provided an evidence bundle, boundary map, examples of one-variable isolation, and explicit changes to avoid prematurely.
Failure point	Could not go beyond the missing reproducer.	Same core limitation: without a real failure, the output remained a debugging framework rather than a root-cause diagnosis.
My preference here	Better when I want a short, strict response that refuses to over-speculate.	Better when I want a reusable investigation checklist and more explicit process detail.

Specific takeaway: both models respected the evidence-first instruction, but they handled missing information differently. Claude stopped earlier; ChatGPT used the same limitation to build a more complete investigation procedure. Neither output should be mistaken for an actual TensorFlow diagnosis because no real failing artifact was included in the test prompt.

Reusable Prompt Template

This is the version I would reuse on a real TensorFlow/Python runtime issue. It keeps the techniques that improved the outputs and adds placeholders for the evidence that was deliberately missing during the six-run exercise.

Act as a senior TensorFlow and Python runtime engineer.

Help me debug the following issue.

Context:

I am working on [TensorFlow version/commit] with [Python version/build]. The problem appears under [normal Python / free-threaded Python / both]. It may involve [native extension code, build compatibility, GIL behavior, synchronization, crash, deadlock, race condition, or incorrect runtime behavior].

Observed symptom:

[Paste the exact error, crash, hang description, wrong result, or import behavior.]

Minimal reproducer:

[Paste the smallest script or command that reproduces the issue.]

Environment:

- OS:
- Python version:
- TensorFlow version/commit:
- Build/install method:
- CPU/GPU:
- Relevant build flags:
- Whether the issue reproduces on another Python version:

Analyze the issue as follows:

1. Classify the failure as import/GIL behavior, build/link failure, native crash, deadlock/hang, race condition, or incorrect result.
2. Identify the most likely Python/C++ or runtime boundary involved.
3. State one primary hypothesis and explain what evidence supports it.
4. Give the smallest test that can falsify that hypothesis.
5. State what code or synchronization changes should NOT be made yet.
6. Only if the hypothesis survives verification, propose the smallest code change.
7. Specify the targeted test and broader regression checks needed to prove the fix.

Use this response format:

- Likely failure mode
- Why it is plausible
- Evidence to collect
- Smallest verification step
- What not to change yet
- Proposed fix, only if verified
- Tests after the fix

- Next action

Do not jump to a patch before the root cause is supported by evidence.

What I learned

Role assignment improved the quality of the intake, but context was what made the conversation technically relevant. Few-shot examples changed the reasoning style, output structure made the result easier to review, and step decomposition made the process safer by preventing a premature patch. The biggest limitation was consistent across the exercise: prompt engineering can improve how evidence is handled, but it cannot invent the missing evidence needed for a real root-cause diagnosis.